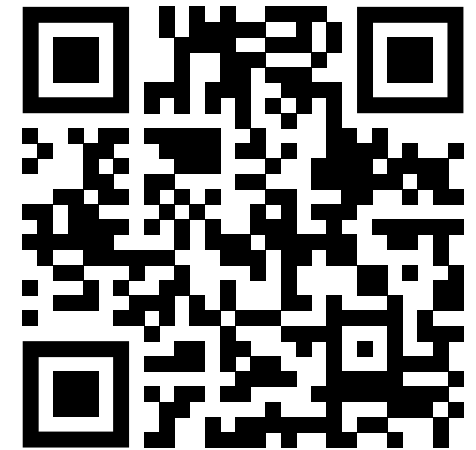


Programmierung in C++

Templates

Prof. Dr. Bernd Lüdemann-Ravit

Ansprechpartner: Dan Eisenkrämer



<https://poll.hs-kempten.de/poll/>



Einführung

Lernziele

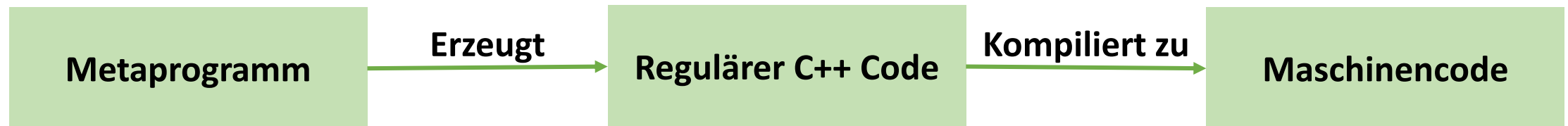
- Die Begriffe Metaprogrammierung und Generische Programmierung können korrekt verwendet werden
- Die Bedeutung und der Nutzen von Templates sind bekannt
 - Der Unterschied zu Objektorientierung wurde verstanden

Einführung

Meta-Programmierung



- C++ Code des Programmierers wird gewöhnlich direkt in Maschinencode übersetzt
- **Metaprogrammierung** ist das Schreiben von Code, der wiederum anderen oder neuen Code erzeugt.
 - Wie Klassen als Schablone für Objekte gesehen werden können, können Meta-Programme als Schablone für „tatsächlich kompilierbaren“ Code gesehen werden.
 - Z. B. ein Python Skript, das anhand von Anforderungen C++ Code generiert
 - Z. B. die Verwendung von Präprozessor Makros



Einführung



Meta-Programmierung - Beispiel

- Ein einfaches Beispiel für Meta-Programmierung sind **Präprozessor-Makros**:
 - Der Präprozessor parst das Makro zur Kompilierzeit und fügt es an den entsprechenden Stellen im Code ein.
 - Dadurch entsteht neuer Code!

```
// Präprozessor-Makro
```

```
#define MAX(a,b) (((a) > (b)) ? (a) : (b))
```

```
int main()
```

```
{
```

```
    int x = 10;
```

```
    int y = 20;
```

```
    std::cout << "Max number of " << x
```

```
        << " and " << y << ": "
```

```
        << MAX(x,y);
```

```
}
```

```
int main()
```

```
{
```

```
    int x = 10;
```

```
    int y = 20;
```

```
    std::cout << "Max number of " << x
```

```
        << " and " << y << ": "
```

```
        << (((x) > (y)) ? (x) : (b));
```

```
}
```

Einführung

Meta-Programmierung



- C++ ermöglicht das Schreiben von Meta-Programmen durch die Verwendung von **Templates** und konstanten Ausdrücken.
 - Der C++ Compiler führt dieses Meta-Programm aus, um den benötigten regulären C++ Code zu generieren und übersetzt diesen dann in Maschinencode.
 - C++ Template Meta-Programmierung besitzt gute Unterstützung zum Analysieren und Manipulieren von C++ Typen
 - Der Metaprogramm-Code ist mit regulärem (nicht-generischen) C++ Code vermischt, was für Anfänger verwirrend sein kann, aber die Verwendung auf Dauer vereinfacht
- Grundsätzlich werden Templates genutzt, um Funktionen, Werte und Klassen zu erzeugen, die unterschiedliche Datentypen unterstützen.
 - Sie verhindern, Code für unterschiedliche Datentypen mehrfach mit geringfügigen Anpassungen schreiben zu müssen.

Einführung

typename



- Das **template<typename T>** Präfix macht **T** zu einem Typparameter für die nachfolgende Funktion (oder Klasse)
 - Kann gelesen werden als „Für alle Typen von T“
 - Statt *typename* kann auch *class* äquivalent verwendet werden
 - sieht man häufiger in älterem Code, da früher nur *class* möglich war

```
/// <summary> Gibt ein Element in einer Konsolenzeile aus
/// </summary>
/// <typeparam name="T">
///     Beliebiger Datentyp, für den << überschrieben ist
/// </typeparam>
/// <param name="output">
///     Element vom Typ T, dass ausgegeben werden soll
/// </param>
template <typename T>
void printlnLine(const T& output) {
    std::cout << output << '\n';
}
```

```
int main() {
    printlnLine("Hallo");
    std::string w{"Welt"};
    printlnLine(w);
    char e = '!';
    printlnLine(e);
    uint16_t y = 2025;
    printlnLine(y);
    Vector v{5, 7, 9};
    printlnLine(v);
}
```

T = const char[6]

T = std::string

T = char

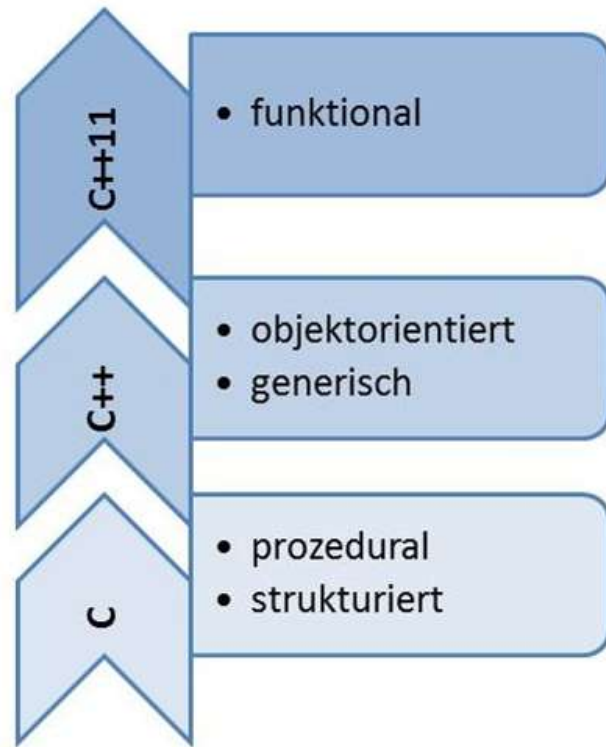
T = uint16_t

T = Vector

```
Hallo
Welt
!
2025
( 5, 7, 9 )
```

Einführung

Objektorientierte vs. generische Programmierung



■ Objektorientierte Programmierung

- Objekt kapselt seine Attribute und Methoden
- Stellt eine Schnittstelle zur Außenwelt dar
- Bietet Vererbung
- Polymorphie findet zur Laufzeit statt

■ Generische Programmierung

- Compiler erzeugt automatisch Code für einen konkreten Typ
- Erlaubt ähnliche Abstraktion wie objektorientierte Programmierung
- Polymorphie findet zur Kompilierzeit statt

■ Funktionale Programmierung

- Funktionen, die andere Funktionen als Argumente erhalten
- Dazu mehr in der nächsten Vorlesung

Einführung

Objektorientierte vs. generische Programmierung



■ Objektorientierte Programmierung

```
class Widget {  
public:  
    Widget();  
    virtual ~Widget();  
    virtual std::size_t size() const;  
    virtual void normalize();  
    void swap(Widget& other);  
    // ...  
};
```

```
void doProcessing(Widget& w) {  
    if (w.size() > 10 && w != someNastyWidget) {  
        Widget temp(w);  
        temp.normalize();  
        temp.swap(w);  
    }  
}
```

- Folgende Aussagen treffen auf **w** zu:
 - **w** ist als **Widget** deklariert
 - **w** muss also zur Widget Schnittstelle passen
 - Die Schnittstelle können wir konkret im Quellcode anschauen
 - Es ist eine **explizite Schnittstelle**
 - Da manche Methoden virtuell sind, wird eine Virtual Method Table angelegt
 - Bei dem Funktionsaufruf virtueller Methoden wird zur Laufzeit entschieden, welche Funktion genau aufgerufen wird
 - **Laufzeit-Polymorphismus**

Einführung

Objektorientierte vs. generische Programmierung



■ Generische Programmierung

■ Folgende Aussagen treffen auf **w** zu:

- Die Schnittstelle, die **w** unterstützen muss, wird anhand der Operationen in dem Template ermittelt
 - Der Datentyp von **w** (T) scheint die Methoden *size*, *normalize* und *swap* besitzen zu müssen
 - T muss *copy constructable* sein und Vergleiche auf Ungleichheit unterstützen
 - Die **implizite Schnittstelle** muss aus dem Code ermittelt werden
- Je nach Typ T können unterschiedliche Realisierungen für `operator!=` notwendig sein
 - **Kompilierzeit-Polymorphismus**

```
template<typename T>
void doProcessing(T& w) {
    if (w.size() > 10 && w != someNastyWidget) {
        T temp(w);
        temp.normalize();
        temp.swap(w);
    }
}
```

Im Vergleich zur Objektorientierung, bei der der Anwender von der expliziten Schnittstelle erben könnte.

Einführung

Objektorientierte vs. generische Programmierung



■ Objektorientierte Programmierung

```
void doProcessing(Widget& w) {  
    if (w.size() > 10 && w != someNastyWidget) {  
        Widget temp(w);  
        temp.normalize();  
        temp.swap(w);  
    }  
}
```

■ explizite Schnittstelle

- Ergibt sich explizit aus dem Quellcode / dem Klassenkörper
- Alle Objekte verwenden die explizite Schnittstelle

■ Laufzeit-Polymorphismus

- Welche Funktion aufgerufen wird, wird zur Laufzeit bestimmt

■ Generische Programmierung

```
template<typename T>  
void doProcessing(T& w) {  
    if (w.size() > 10 && w != someNastyWidget) {  
        T temp(w);  
        temp.normalize();  
        temp.swap(w);  
    }  
}
```

■ implizite Schnittstelle

- Ergibt sich implizit aus den Anforderungen aus einer generischen Funktion
- Alle Datentypen für T müssen zur impliziten Schnittstelle passen

■ Kompilierzeit-Polymorphismus

- Welche Funktion aufgerufen wird, wird zur Kompilierzeit bestimmt



- **Meta-Programmierung** ist das Schreiben von Code, der wiederum anderen oder neuen Code erzeugt.
 - Bei der **generischen Programmierung** erzeugt der Compiler automatisch Code für einen konkreten Typ
 - Grundsätzlich werden Templates genutzt, um **Funktionen**, **Werte** und **Klassen** zu **erzeugen**, die unterschiedliche Datentypen unterstützen.
 - Generische Programmierung wird mit dem **template** Schlüsselwort eingeleitet
 - **typename** führt einen Platzhalter für beliebige Datentypen ein
- **Objektorientierung vs. generische Programmierung**
 - Bei **Objektorientierung** existiert eine **explizite Schnittstelle** (der Klassenkörper)
 - Bei **generischer Programmierung** ergibt sich die **Schnittstelle implizit**
 - **Objektorientierung** nutzt Virtual Method Tables, um den Funktionsaufruf zur **Laufzeit** zu bestimmen
 - Bei **generischer Programmierung** wird der Funktionsaufruf zur **Kompilierzeit** bestimmt

Einführung

Verständnisfragen

PIN: 862985

<https://poll.hs-kempten.de/poll/>



■ Was ist ein Template?

- Eine Klasse, die ausschließlich rein virtuelle Methoden besitzt.
- Ein temporäres Objekt, das später gelöscht wird.
- Ein Mechanismus, um Datentypen als Parameter verwenden zu können.
- Eine Blaupause, aus der der Compiler konkrete Klassen und Funktionen erzeugt.

■ WAHR oder FALSCH?

- Template-Metaprogrammierung erfolgt zur Kompilierzeit.
- Templates ermöglichen, dass einer Variable kein fester Datentyp zugeordnet ist.
- Templates ermöglichen die Erstellung generischer Funktionen und Klassen.
- Templates ermöglichen das Schreiben von Code, der für verschiedene Datentypen wiederverwendet werden kann.
- Metaprogrammierung kann den Kompilierzeitaufwand erhöhen, hat jedoch keinen Einfluss auf die Laufzeitleistung.



Parametrisierte Operationen

Lernziele

- Funktionen können parametrisiert werden
 - Mithilfe von Funktionsüberladungen können die parametrisierten Operationen spezialisiert werden.
- Variadische Templates sind bekannt und können geschickt eingesetzt werden.

Parametrisierte Operationen



Funktions-Templates

- Der Compiler **realisiert** den konkreten Code für die benötigten Template-Parameter
 - Anhand der Angabe im Funktionsaufruf ermittelt der Compiler, für welche **Realisierungen** der Code erzeugt werden muss
 - **C++17** **Template Argument Deduction**: Anhand der Parameterliste kann der Compiler automatisch deduzieren, welche Realisierungen benötigt werden

```
// pow_n akzeptiert jeden Zahlentyp
template <typename T>
auto pow_n(const T& v, int n) {
    auto product = static_cast<T>{1};
    for (int i = 0; i < n; ++i) {
        product *= v;
    }
    return product;
}
```

speichert die 1
im Datentyp der
Realisierung ab

```
auto x = pow_n<float>(2.0f, 3); // x ist float
auto y = pow_n<int>(3, 3);      // y ist int
```

Weist den Compiler an, eine
Realisierung für **int** zu erzeugen

```
C++17
auto x = pow_n(2.0f, 3); // x ist float
auto y = pow_n(3, 3);    // y ist int
```

// Realisierung für int

```
int pow_n(const int& v, int n) {
    int product = static_cast<int>{1};
    for (int i = 0; i < n; ++i) {
        product *= v;
    }
    return product;
}
```

Parametrisierte Operationen

Wert Templates



- Neben Datentyp-Argumenten können Templates auch **Wert Argumente** besitzen
 - Der Compiler erzeugt eine neue Funktion für jeden einzigartigen Wert, der verwendet wird
- Wert Argumente müssen eine konstante Größe und ein konstantes Verhalten besitzen
 - Es darf nicht auf temporäre Objekte verwiesen werden, da diese nicht zur Compile-Zeit bekannt sind. (z.B. Pointer/Referenzen auf Objekte)
 - String Literale (z.B. `std::string`) können aufgrund der flexiblen Größe nicht als Wert Argument verwendet werden

```
template <int N, typename T>
auto const_pow_n(const T& v) {
    auto product = static_cast<T>(1);
    for (int i = 0; i < N; ++i)
        product *= v;
    return product;
}

auto x2 = const_pow_n<2>(4.0f); // Quadrat
auto x3 = const_pow_n<3>(3.1f); // Cubik
```

Der Compiler wird 2 Funktionen erzeugen:

- Für die Realisierung mit `<2,float>`
- Für die Realisierung mit `<3,float>`

Parametrisierte Operationen



Default-Werte

- Wie auch bei Funktionsparametern können bei der Deklaration von Template-Parametern Default Werte angegeben werden.
 - Wenn für einen Template-Parameter ein Default Wert spezifiziert wird, müssen auch für alle nachfolgenden Template-Parameter Default Werte angegeben werden.
 - Für Template-Parameter mit Default Werten muss bei der Realisierung kein Template-Argument angegeben werden. (Gleiches Verhalten wie bei Funktionsparametern)

```
template <int N = 2, typename T = int>
auto const_pow_n(const T& v) {
    auto product = static_cast<T>(1);
    for (int i = 0; i < N; ++i)
        product *= v;
    return product;
}
```

Für N wird der Default Wert 2 genommen.
Für T wird anhand des Parameters float deduziert.

```
auto x2 = const_pow_n(4.0f); // Quadrat <2,float>
auto x3 = const_pow_n<3>(3.1f); // Cubik <3,float>
```

Parametrisierte Operationen

Beispiel

- Wir möchten eine Funktion schreiben, die alle Elemente in einem Container auf einen Wert aufsummiert und zurückgibt.
 - Der Datentyp eines Initialwertes **init** legt auch den Rückgabe-Wert der Funktion fest
 - Wir nehmen an, dass der Container einen range-based For-Loop unterstützt



```
template<typename Sequence, typename Value>
Value sum(const Sequence& s, Value init) {
    for (auto x : s)
        init += x;
    return init;
}
```

Das „Value“ Template Argument und Funktionsargument „init“ erlauben dem Aufrufer, den Datentyp und initialen Wert des Akkumulators festzulegen.

Die Typen für „Sequence“ und „Value“ werden beim Aufruf automatisch deduziert.

```
void example(Vector<int>& vi, std::list<double>& ld, std::set<std::string>& ss) {
    int x = sum(vi, 0);           // Summe der Integer (Integer werden addiert)
    double d = sum(vi, 0.0);      // Summe der Integer (Doubles werden addiert)
    double dd = sum(ld, 0.0);     // Summe der Doubles (Doubles werden addiert)
    auto s = sum(ss, "");        // Konkatenation der Strings
}
```



Parametrisierte Operationen

Spezialisierung für Funktionen – Motivation

Sequence	std::vector<int>
Value	int

- Addition der Ganzzahlen
- Rückgabe als *int*

Sequence	std::list<double>
Value	int

- *double* wird implizit in *int* umgewandelt
- Rückgabe als *int*

Sequence	Vector<std::string>
Value	std::string

- Konkatenation der Strings
- Rückgabe als *std::string*

```
// Default-Verhalten für alle Typen
// außer den Überladenen
template<typename Sequence, typename Value>
Value sum(const Sequence& s, Value init) {
    for (auto x : s)
        init += x;
    return init;
}
```

Durch das Default-Verhalten wird für Strings **immer eine Konkatenation** durchgeführt!

Wir gehen nun aber davon aus, dass in den Strings Zahlen in unterschiedlichen Basen (Dezimal, Hexadezimal,...) kodiert sind. **Diese Zahlen möchten wir addieren und am Ende wieder als String ausgeben.**

Parametrisierte Operationen



Spezialisierung für Funktionen

- Um Funktions-Templates für bestimmte Datentypen zu **spezialisieren**, werden sie **wie normale Funktionen überladen**.
 - **Spezialisierung** bedeutet, für Spezialfälle/spezielle Realisierungen eine andere Funktion anzugeben
 - z.B. wenn nicht gewünscht ist, dass **sum()** für Strings eine Konkatination ausführt.

```
// Default-Verhalten für alle Typen
// außer den Überladenen
template<typename Sequence, typename Value>
Value sum(const Sequence& s, Value init) {
    for (auto x : s)
        init += x;
    return init;
}
```

```
int main() {
    std::list<std::string> strings = { "-10", "010", "0xF0" };
    auto res = sum(strings, "0"s); // res ist "238"
}
```

```
// Überladung, wenn Value ein std::string ist
template<typename Sequence>
std::string sum(const Sequence& s, std::string init) {
    // Geh davon aus, dass im String ein Zahlenwert kodiert ist
    auto number = std::stol(init, nullptr, 0);
    // Geh davon aus, dass die Sequenz aus Strings besteht
    for (std::string x : s)
        number += std::stol(x, nullptr, 0);
    return std::to_string(number);
}
```

-10 (dezimal) + 10 (oktal) + F0 (hexadezimal)
= -10 + 8 + 240 = 238

std::stol() versucht ein String in ein *long* umzuwandeln. Durch die 0 wird automatisch die Basis des Zahlenwertes ermittelt.



Parametrisierte Operationen

Spezialisierung für Nicht-Funktionsparameter

- Entsteht durch das Überladen keine eindeutige Spezialisierung, muss eine **Spezialisierung für alle Template-Parameter** explizit angegeben werden
 - Z.B. wenn ein Template-Parameter nicht in der Funktions-Parameterliste vorkommt
 - Eine partielle Spezialisierung der Template-Parameter ist bei Funktionen nicht möglich

```
template <int N, typename T>
auto const_pow_n(const T& v) {
    auto product = static_cast<T>(1);
    for (int i = 0; i < N; ++i)
        product *= v;
    return product;
}
```

Spezialisierung für N = 2

Wir müssen auch
typename T spezialisieren

Wir wollen für N=2 eine
einfachere Funktion verwenden

```
template <>
auto const_pow_n<2, int>(const int& v)
{
    return v * v;
}
```

Es handelt sich um ein Template,
bei dem alle Template-Parameter
spezialisiert sind.

Die Spezialisierungen sind
explizit nach dem
Funktionsnamen angegeben.

Wenn möglich sollte die Funktions-Überladung der
Funktions-Spezialisierung bevorzugt werden!

Parametrisierte Operationen

Compiler-Verhalten



```
template <typename T>
auto pow_n(const T& v, int n) {
    // ...
}
```

erzeugt <int>

erzeugt <float>

```
template <int N, typename T>
auto const_pow_n(const T& v) {
    // ...
}
```

erzeugt <2,float>

erzeugt <3,float>

erzeugt <2,int>

```
template <>
auto const_pow_n<2, int>(const int& v) {
    // ...
}
```

```
auto a = pow_n(42, 3);           // Generate new function
auto b = pow_n(42.f, 2);        // Generate new function
auto c = pow_n(17.f, 5);
auto d = const_pow_n<2>(42.f);  // Generate new function
auto e = const_pow_n<2>(99.f);
auto f = const_pow_n<3>(42.f);  // Generate new function
auto g = const_pow_n<2>(3);     // Generate new function
```

Compiler erzeugt

```
auto pow_n__float(float v, int n) { /*...*/ } // Genutzt von: a
auto pow_n__int(int v, int n) { /*...*/ }     // Genutzt von: b,c
auto const_pow_n__2_float(float v) { /*...*/ } // Genutzt von: d,e
auto const_pow_n__3_float(float v) { /*...*/ } // Genutzt von: f
auto const_pow_n__2_int(int v) { /*...*/ }     // Genutzt von: g
```

Parametrisierte Operationen



Verkürzte Schreibweise für Funktions-Templates

- Mithilfe von **auto** für Funktions-Parametertypen lässt sich eine parametrisierte Funktion in verkürzter Schreibweise erstellen
 - Ähneln damit mehr der Schreibweise, die wir bei Lambda-Ausdrücken kennen lernen werden
 - Durch diese Schreibweise gibt es keinen konkreten Platzhalter (**T**) mehr

statt

```
template <typename T>
auto pow_n(const T& v, int n) {
    // ...
}
```

verkürzte
Schreibweise:

```
auto pow_n(const auto& v, int n) {
    // ...
}
```

neue Herausforderung:

```
auto pow_n(const auto& v, int n) {
    auto product = static_cast<T>(1);
    for (int i = 0; i < n; ++i) { product *= v; }
    return product;
}
```

ERROR: Es gibt keinen
konkreten Platzhalter T

Lösung mit **decltype**:

```
auto pow_n(const auto& v, int n) {
    auto product = static_cast<decltype(v)>(1);
    for (int i = 0; i < n; ++i) { product *= v; }
    return product;
}
```

Parametrisierte Operationen



C++11 Decltype

- **decltype** ermittelt den Datentyp einer Variable
 - Wird genutzt, wenn der konkrete Typ-Name nicht verfügbar ist
 - **decltype** leitet auch **const**, **&** und ***** ab

Kann entfallen

Beispiel für `pow_n<int>(3, 3);`

`(const int& v, int n)`

```
auto pow_n(const auto& v, int n) {  
    decltype(v) product{1};  
    for (int i = 0; i < n; ++i) {  
        product *= v; // Error!  
    }  
    return product;  
}
```

`const int& product{1};`

product ist eine non-modifiable L-Value-Reference!

```
auto pow_n(const auto& v, int n) {  
    auto product = static_cast<decltype(v)>(1);  
    for (int i = 0; i < n; ++i) { product *= v; }  
    return product;  
}
```

auto leitet die Referenz nicht ab

v ist `const X&`, wobei X je nach Realisierung ersetzt wird

Bei der Verwendung von **decltype** entstehen neue Herausforderungen.

Parametrisierte Operationen



C++20 Remove const-value-reference

```
#include <type_traits>
```

- **C++11** Der ***type_traits*** Header erweitert die Metaprogrammierung
 - Geht grundsätzlich über den Umfang dieser Vorlesung hinaus
 - **C++20** Stellt die Funktion **`remove_cvref`** zur Verfügung, mit der die automatische Deduktion von ***const &*** durch ***decltype*** entfernt werden kann.

```
typename std::remove_cvref<decltype(v)>::type product{1};
```

Spezifiziert, dass es sich bei dem Member **`::type`** um einen **Typ-Name** (Typ-Alias) handelt.

Wird mit dem deduzierten Datentyp von **`v`** realisiert.

type erhält den Datentypen ohne die **const-value-Reference**

Beispiel für **`pow_n<int>(3, 3);`**

`(const int& v, int n)`

```
auto pow_n(const auto& v, int n) {  
    typename std::remove_cvref<decltype(v)>::type product{1};  
    for (int i = 0; i < n; ++i) { product *= v; }  
    return product;  
}
```

product ist ein modifiable L-Value
=> Die Funktion funktioniert

`decltype(v) -> const int&`

`type -> int`

Lessons Learned:

1. Die verkürzte Schreibweise mit ***auto*** reduziert nicht immer den Aufwand.
2. Es gibt Konzepte der Metaprogrammierung (z.B. ***type traits***) die weit über Umfang dieser Vorlesung hinausgehen!

Parametrisierte Operationen



C++11 Variadische Templates

- **Variadische Templates** erlauben, für ein Template beliebig viele Werte als Argumente an ein Template zu übergeben.
 - Dazu wird ein Parameter mit einem ... zu einem **Parameterpaket** deklariert.
 - Ein Parameterpaket kann unterschiedliche Datentypen zulassen (gekennzeichnet durch *typename*) oder von einem bestimmten Datentyp sein (konkrete Angabe des Datentyps).
 - Die Elemente des Parameterpakets können durch rekursive Funktionsaufrufe abgearbeitet werden.
 - Bieten eine große Flexibilität, die besonders in der Standardbibliothek häufig verwendet wird

```
void print() { }
template <typename T>
void print(const T& t) {
    std::cout << t << '\n';
}
template <typename First, typename... Rest>
void print(const First& first, const Rest&... rest) {
    std::cout << first << ", ";
    // Rekursiver Aufruf mit Parameterpaket
    print(rest...);
}
int main() {
    print();
    print(1);
    print(10, 20);
    print(100, 200, 300);
    print("first", 2, "third", 3.14159);
}
```

Parametrisierte Operationen



C++17 Variadische Templates

- **Fold Expressions** vereinfachen die Implementierung variadischer Templates
 - Erlauben für jeden binären Operator, die Operation über alle Elemente des Parameterpakets auszuführen
- 1) (**pack op ...**) wendet die Operation als „**right fold**“ an, also von hinten nach vorne:
(**pack₀ op (... op (pack_{n-2} op pack_{n-1})))**
- 2) (**pack op ... op init**) wendet die Operation als „**right fold**“ mit einem Initialwert an
- 3) (... **pack op**) wendet die Operation als „**left fold**“ an, also von vorne nach hinten:
(((**pack₀ op pack₁**) op ...) op **pack_{n-1}**)
- 4) (**init op ... op pack**) wendet die Operation als „**left fold**“ mit einem Initialwert an

```
template<typename Value, typename... Values>
Value sum(Value init, Values... v) {
    return (v + ... + init); // Right Fold
}

template<typename... Value>
void print(Value&&... args) {
    (std::cout << ... << args) << '\n'; // Left Fold
}

int main() {
    print(
        sum(5, 2.4, 97), '\n',
        sum('a', 5, 2.4), '\n',
        sum(2.4, 'a', 5), '\n',
        sum("a"s, "2.4", "5", "...")
    );
}
```

```
104
h
104.4
2.45...a
```

Parametrisierte Operationen



C++11 Universelle Referenzen

- Eine Variable oder ein Parameter mit deklariertem Typ **T&&** für einen **deduzierten Datentyp T** ist eine **Universelle Reverenz**
 - Universelle Referenzen treten meistens bei **auto**-deklarierten Variablen und **Parametern von Funktions-Templates** auf
 - Eine Universelle Reverenz bindet ein **L-Value** als **L-Value-Referenz** und ein **R-Value** als **R-Value-Referenz**
 - Wird daher auch „**Forwarding Reference**“ genannt

```
template<typename... Value>
void print(Value&&... args) {
    (std::cout << ... << args) << '\n';
}
```

```
int main() {
    double d{ 3.0 };
    const char c{ '!' };
    print(5, 'a', "test"s, d, c);
}
```

int&&

char&&

double&

std::string&&

const char&

```
std::vector<int> v;
```

```
/* ... */
```

```
auto&& val{ v[0] }; // val => lvalue reference (int&)
```

```
auto&& v2{ v }; // v2 => lvalue reference (std::vector<int>&)
```

```
auto&& v3{std::move(v)}; // v3 => rvalue reference (std::vector<int>&&)
```

Parametrisierte Operationen

C++11 std::forward

- **std::forward** gibt einen Funktionsparameter als den Wertausdruck weiter, den der Parameter bei Aufruf der Funktion hatte.
 - Für einen Template-Parameter T und Funktions-Parameter $T\&\& t$ gibt `std::forward` **`static_cast<T\&\&>(t)`** zurück.
 - Wenn eine Template-Funktion seine Argumente weitergibt, ohne deren Wertausdruck zu ändern, wird dies als „**Perfect Forwarding**“ bezeichnet.



```
#include <utility>
```

```
template <typename T>
void print(T\&\& t) {
    std::cout << t << '\n';
}

template <typename First, typename... Rest>
void print(First\&\& first, Rest\&\&... rest) {
    std::cout << first << ", ";
    // Rekursiver Aufruf mit Parameterpaket
    print(std::forward<decltype(rest)>(rest)...);
}

int main() {
    double d{ 3.0 };
    const char c{ '!' };
    print(5, 'a', "test"s, d, c);
}
```

rest ist ein L-Value!

rest wird wieder in den Wertausdruck umgewandelt, den er bei Aufruf der Funktion hatte.

```
print(static_cast<char\&\&>(_0), static_cast<std::string\&\&>(_1), static_cast<double\&>(_2), static_cast<const char\&>(_3));
```

Parametrisierte Operationen



Zusammenfassung

- Mithilfe von Template-Parametern können die **gleichen Funktionalitäten** für unterschiedliche Datentypen verwendet werden
 - Anhand des Funktionsaufrufs **realisiert** der **Compiler** den notwendigen Funktionscode
- Template-Parameter können wie Funktions-Parameter **Default-Werte** besitzen.
- Um Funktions-Templates für bestimmte Datentypen zu **spezialisieren**, werden sie wie normale Funktionen **überladen**.
 - Spezialisierung bedeutet, einen Template-Parameter für Spezialfälle/spezielle Werte konkret festzulegen
- **Variadische Templates** ermöglichen, dass Funktionen eine beliebige Anzahl an Parametern übergeben bekommen können.
 - Die Parameter in einem **Parameterpaket** können beliebige Datentypen besitzen oder auf einen konkreten Datentyp festgesetzt werden.
 - **Fold Expressions** ermöglichen kompakt, binäre Operationen über alle Parameter in einem Parameterpaket auszuführen.

Parametrisierte Operationen

Verständnisfragen

PIN: 862985

<https://poll.hs-kempten.de/poll/>



■ Wofür nutzt man das Schlüsselwort `decltype` in C++?

- Um zur Laufzeit den Typ einer Variable zu ermitteln.
- Um zur Kompilierzeit sicherzustellen, dass zwei Typen identisch sind.
- Um einen Typ zu bestimmen, ohne den Ausdruck direkt auswerten zu müssen.
- Um eine Typumwandlung eines Ausdrucks auf einen bestimmten Typ zu erzwingen.

■ WAHR oder FALSCH?

- Generalisierte Funktionen können nicht überladen werden.
- Template-Parameter können sowohl mit einfachen Datentypen als auch benutzerdefinierten Typen spezialisiert werden.
- Ein vollständig spezifiziertes Template kann als normale Funktion betrachtet werden.
- Variadische Templates erlauben es, eine beliebige Anzahl von Argumenten in einer Template-Deklaration zu akzeptieren.
- Variadische Templates erfordern mindestens zwei Typ-Parameter.



Parametrisierte Typen

Lernziele

- Mithilfe von Templates können eigene Klassen und Strukturen für beliebige Datentypen ausgelegt werden.
- Die grundsätzlichen Möglichkeiten und Herausforderungen mit parametrisierten Typen sind bekannt.

Parametrisierte Typen

Einführendes Beispiel



- Wir haben bereits Containerklassen kennengelernt, die Elemente eines bestimmten Datentyps verwalten.
 - Der Datentyp kann dabei beliebig sein und ist zur Kompilierzeit festgesetzt.
 - Die Funktionalität der Containerklassen ist **unabhängig** von dem Typ der Elemente.
- Die Definitionen aller Member-Funktionen einer Template-Klasse müssen mit **demselden Präfix** gekennzeichnet sein
- Durch Angabe des gewünschten Datentyps nach dem Klassennamen kann eine Realisierung der Klasse instanziiert werden

```
Vector<char> vc(200);           // Vector mit 200 Zeichen
Vector<std::string> vs(17);     // Vector mit 17 Strings
Vector<std::list<int>> vli(45);  // Vector mit 45 Integer-Listen
```

```
template<typename T>
class Vector {
public:
    explicit Vector(std::size_t size);
    ~Vector();
    std::size_t getSize() const;
    T& operator[] (std::size_t idx);
private:
    T* m_elements;
    std::size_t m_size;
};
```

```
template<typename T>
Vector<T>::Vector(std::size_t size)
    : m_size{size}
    , m_elements{new T[size]}
{ }
```

Parametrisierte Typen

Kompilieren & Linken



- Die Definition von Templates lässt sich **nicht** ohne Weiteres von ihrer Deklaration trennen und in eine .cpp Datei packen!
 - Ein Template ist selbst keine Klasse oder Funktion, sondern ein „Pattern“, welches vom Compiler genutzt wird, um eine Familie von Klassen oder Funktionen zu erzeugen.
 - Damit der Compiler die Familie erzeugen kann, muss er sowohl Einsicht auf die Template Definition erhalten als auch **auf alle Realisierungen** des Templates.
 - Der Compiler berücksichtigt nicht den Inhalt einer .cpp Datei, während er eine andere .cpp Datei kompiliert. Wenn sich die Template Definition in einer .cpp Datei befindet, weiß der Compiler nicht, für welche Realisierungen aus anderen .cpp Dateien er die konkrete Klasse oder Funktion erstellen soll.

```
template<typename T> Foo.h
class Foo {
public:
    Foo();
    void someMethod(T x);
    T x;
};
```

```
Foo.cpp
#include "Foo.h"
template<typename T>
Foo<T>::Foo() { /*...*/ }
template<typename T>
void Foo<T>::someMethod(T x) { /*...*/ }
```

```
Foo.cpp
#include "Foo.h"
int main(){
    Foo<int> f;
    f.someMethod(5);
}
```

LNK2019: Verweis auf nicht aufgelöstes externes Symbol

""public: __cdecl Foo<int>::Foo<int>(void)" (??0?\$Foo@H@@QEAA@XZ)" in Funktion "main".

Parametrisierte Typen

Kompilieren & Linken



- Um das Problem mit dem Linker-Fehler zu beheben, gibt es zwei sinnvolle Lösungen
 1. Wenn die **Definition zusammen mit der Deklaration in der Header-Datei** steht, entstehen die Linker-Fehler nicht. Diese Lösung führt unter Umständen zu einer deutlichen Verlängerung der Kompilierzeit und Vergrößerung der Executable-Datei (oder auch nicht, wenn der Compiler klug genug ist).
 2. Alternativ können die Definition in der .cpp und die Deklaration in der .h Datei bleiben. Dann müssen **in der .cpp Datei alle benötigten Realisierungen des Templates** konkret angegeben werden:

```
template<typename T>
class Foo {
public:
    Foo();
    void someMethod(T x);
    T x;
};
```

Foo.h

```
#include "Foo.h"
template<typename T>
Foo<T>::Foo() { /*...*/ }
template<typename T>
void Foo<T>::someMethod(T x) { /*...*/ }

template class Foo<int>;
template class Foo<std::string>;
template class Foo<double>;
```

Foo.cpp

```
#include "Foo.h"
int main(){
    Foo<int> f;
    f.someMethod(5);
}
```

Parametrisierte Typen

Parametrisierte Member-Funktionen

- **Member-Funktionen** können genau wie andere Funktionen parametrisiert werden
 - Dazu wird die Template-Spezifikation im Klassenkörper vor der Methode angegeben
- *Beispiel:* Wir wollen für eine eigene Smart-Pointer Klasse implizite Typumwandlung (z.B. für Up-Casts) unterstützen

```
class Top { /* ... */ };
class Middle: public Top { /* ... */ };
class Bottom: public Middle { /* ... */ };
class Left { /* ... */ };
SmartPointer<Top> pt1 = SmartPtr{new Middle}; // konvertiert Middle* -> Top*
SmartPointer<Top> pt2 = SmartPtr{new Bottom}; // konvertiert Bottom* -> Top*
SmartPointer<const Top> pct2 = pt1; // konvertiert Top* -> const Top*
// Compiler-Fehler:
SmartPointer<Left> pt3 = pt2; // Typumwandlung von Top* -> Left* nicht möglich!
```



```
template<typename T>
```

```
class SmartPtr {
```

```
public:
```

```
template<typename U>
```

```
SmartPtr(const SmartPtr<U>& other);
```

```
T* get() const;
```

```
// weitere Konstruktoren, Destruktor, ...
```

```
private:
```

```
T* m_heldPtr;
```

```
};
```

```
template<typename T>
```

```
template<typename U>
```

```
SmartPointer<T>::SmartPointer(const SmartPtr<U>& other)
```

```
: m_heldPtr{other.get() }
```

```
{ /* ... */ }
```

```
template<typename T>
```

```
T* SmartPtr<T>::get() const {
```

```
return m_heldPtr;
```

```
}
```

Copy-Konstruktor ist für beliebige von SmartPtr gehaltene Typen generalisiert

Erst wird die Generalisierung der Klasse und dann die der Methode angegeben.

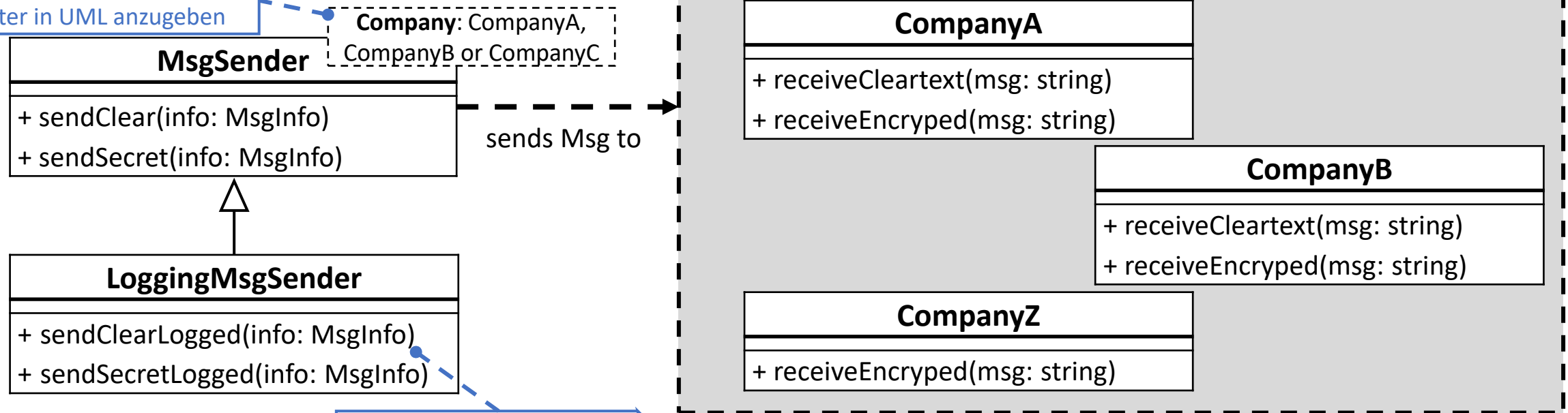
Parametrisierte Typen



Beispiel

- Instanzen der Klasse **MsgSender** senden Nachrichten entweder verschlüsselt oder unverschlüsselt an ein beliebiges Unternehmen (**Company**)
 - Zur **Kompilierzeit** ist bekannt, an welches Unternehmen die Nachrichten geschickt werden
 - **CompanyZ** akzeptiert nur verschlüsselte Kommunikation
 - Die Kindklasse **LoggingMsgSender** schreibt zusätzlich Informationen in eine Datei

Möglichkeit, um Template-Parameter in UML anzugeben



MsgInfo sei eine Klasse, die Informationen für die Nachricht enthält.



Parametrisierte Typen

Beispiel – Vollständige Spezialisierung

```
template<typename Company>
class MsgSender {
public:
    // ctors, dtor, etc.
    void sendClear(const MsgInfo& info) {
        std::string msg;
        // erzeuge den msg-String aus info;
        Company c;
        c.receiveCleartext(msg);
    }
    void sendSecret(const MsgInfo& info)
    { /* ähnlich wie sendClear, aber ruft receiveEncrypted */ }
};
```

```
template<>
class MsgSender<CompanyZ> {
public:
    // Gleich wie das Default-Template, aber ohne sendClear
    void sendSecret(const MsgInfo& info)
    { /* ... */ }
};
```

Vollständige Spezialisierung
von MsgSender für
Company=CompanyZ

```
class CompanyA {
public:
    // ...
    void receiveCleartext(const std::string& msg);
    void receiveEncrypted(const std::string& msg);
    // ...
};

class CompanyB {
public:
    // ...
    void receiveCleartext(const std::string& msg);
    void receiveEncrypted(const std::string& msg);
    // ...
};

class CompanyZ {
public:
    // ...
    void receiveEncrypted(const std::string& msg);
    // ...
};
```



Parametrisierte Typen

Beispiel – Vererbung (Problemstellung)

Der Template-Parameter muss bei der Vererbung angegeben werden.

```
template<typename Company>
class LoggingMsgSender : public MsgSender<Company>
{
public:
    // ctors, dtor, etc.
    void sendClearLogged(const MsgInfo& info)
    {
        // schreib "before sending" info in Log-Datei;
        sendClear(info); // ruf Funktion der Elternklasse
        // schreib "after sending" info in Log-Datei;
    }
};
```

Der Code kompiliert **nicht!**
Der tatsächliche Code der Elternklasse existiert (noch) nicht, nur der Metaprogramm-Code!
-> Der Compiler schaut nicht in der Elternklasse, ob die Funktion sendClear existiert

```
template<typename Company>
class MsgSender {
public:
    // ctors, dtor, etc.
    void sendClear(const MsgInfo& info) {
        std::string msg;
        // erzeuge den msg-String aus info;
        Company c;
        c.receiveCleartext(msg);
    }
    void sendSecret(const MsgInfo& info)
    { /* ähnlich wie sendClear, aber ruft receiveEncrypted */ }
};
```

```
template<>
class MsgSender<CompanyZ> {
public:
    // Gleich wie das Default-Template, aber ohne sendClear
    void sendSecret(const MsgInfo& info)
    { /* ... */ }
};
```

Im Fall Company=CompanyZ würde die Funktion tatsächlich nicht existieren!

Parametrisierte Typen

Beispiel – Vererbung (Problemlösung)



■ Lösung 1: This-Pointer

```
template<typename Company>
class LoggingMsgSender
: public MsgSender<Company> {
public:
    // ...
    void sendClearLogged
        (const MsgInfo& info) {
        // schreib "before sending" info
        this->sendClear(info);
        // schreib "after sending" info
    }
};
```

- Gibt dem Compiler an, er soll davon ausgehen, dass sendClear vererbt ist

■ Lösung 2: Using

```
template<typename Company>
class LoggingMsgSender
: public MsgSender<Company> {
public:
    using MsgSender<Company>::sendClear;
    // ...
    void sendClearLogged
        (const MsgInfo& info) {
        // schreib "before sending" info
        sendClear(info);
        // schreib "after sending" info
    }
};
```

- Weist den Compiler explizit darauf hin, dass die Funktion in der Elternklasse existiert

■ Lösung 3: Expliziter Aufruf

```
template<typename Company>
class LoggingMsgSender
: public MsgSender<Company> {
public:
    // ...
    void sendClearLogged
        (const MsgInfo& info) {
        // schreib "before sending" info
        MsgSender<Company>::sendClear(info);
        // schreib "after sending" info
    }
};
```

- Ruft explizit die Funktion sendClear aus der Elternklasse auf
 - Wenn die Funktion virtuell ist, wird damit jedoch das Virtual Method Binding umgangen!

Alle Lösungen **versprechen** dem Compiler, dass die Realisierung der Elternklasse die benötigte Schnittstelle unterstützen wird.

Parametrisierte Typen

Beispiel – Anwendung



```
MsgInfo msgData;  
// put info in msgData
```

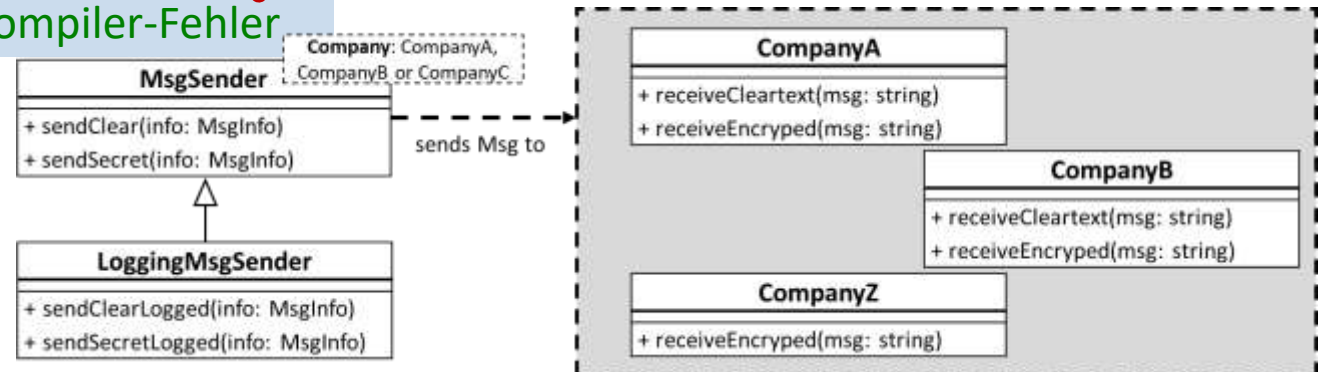
```
MsgSender<CompanyA> aMsgSender;  
aMsgSender.sendEncrypted(msgData); // funktioniert
```

```
LoggingMsgSender<CompanyB> bMsgSender;  
bMsgSender.sendClearLogged(msgData); // funktioniert
```

```
LoggingMsgSender<CompanyZ> zMsgSender;  
zMsgSender.sendEncrypted(msgData); // funktioniert  
zMsgSender.sendClearLogged(msgData); // Compiler-Fehler
```

```
template<>  
class MsgSender<CompanyZ> {  
public:  
    // Gleich wie das Default-Template, aber ohne sendClear  
    void sendSecret(const MsgInfo& info)  
    { /* ... */ }  
};
```

Das Versprechen, dass sendClear() in der Elternklasse existiert wird aufgrund der Spezialisierung für CompanyZ an dieser Stelle gebrochen.



Parametrisierte Typen

partielle Spezialisierung



- Für Typen in C++ können einzelne Template-Parameter spezialisiert werden:

Partielle Spezialisierung

- Der Template-Parameter, der konkret angegeben wird, wird nicht mehr in der Template-Parameter-Liste angegeben, sondern stattdessen nach dem Typ-Namen

Die erste Template-Definition im Code muss das Default-Verhalten implementieren.

```
// Default Template
template<typename T1, typename T2, int I>
class A {};

template<typename T, int I>
class A<T, T*, I> {}; // Partielle Spezialisierung: T2 ist Pointer auf T1

template<typename T, typename T2, int I>
class A<T*, T2, I> {}; // Partielle Spezialisierung: T1 ist ein Pointer

template<typename T>
class A<int, T*, 5> {}; // Partielle Spezialisierung:
                       // T1 ist int, I ist 5, T2 ist ein Pointer
```

Parametrisierte Typen



Zusammenfassung

- Template-Parameter können **anstelle eines konkreten Datentyps** für eigene Klassen und Strukturen verwendet werden.
 - Dadurch kann die **gleiche Implementierung für beliebige Datentypen** verwendet werden.
 - Der Compiler erstellt für die tatsächlich verwendeten Datentypen die benötigten Objektdaten.
 - Dazu müssen die Deklarationen und Definitionen der parametrisierten Typen zusammen in der Header-Datei sein **oder** die verwendeten Datentypen explizit zusammen in der Quellcodedatei mit den Definitionen angegeben werden.
- Wird für bestimmte Template-Argumente ein spezielles Verhalten benötigt, können **vollständige und partielle Spezialisierungen** implementiert werden.

Parametrisierte Typen

Verständnisfragen

PIN: 862985

<https://poll.hs-kempten.de/poll/>



■ WAHR oder FALSCH?

- Templates erfordern die Angabe aller unterstützten Typen bei der Deklaration.
- Templates erfordern, dass der komplette Code für alle möglichen Typen zur Kompilierzeit erzeugt wird.
- Eine Template-Klasse kann mit mehreren Typ-Parametern definiert werden.
- Es ist möglich, für eine Template-Klasse Standardwerte für Typ-Parameter anzugeben.
- Eine spezialisierte Version einer Template-Klasse überschreibt die allgemeine Template-Definition vollständig.
- Eine Template-Klasse kann nicht von einer anderen Template-Klasse erben.
- Template-Klassen können nur mit Typen instanziiert werden, die Standard-Konstruktoren haben.



Ausblick: Meta-Programmierung

Lernziele

- Weiterführende Konzepte der Meta-Programmierung in C++ sind bekannt.
- Als ein Ausblick zur Meta-Programmierung werden Type Traits vorgestellt.

Ausblick: Meta-Programmierung



Problemstellung

- Mithilfe von Templates können wir generische Klassen und Funktionen schreiben, die für jeden Datentypen funktionieren.
 - **Problem:** Wir können nicht direkt **Restriktionen/Bedingungen an die Datentypen** stellen, die das Template realisieren können.
 - **Lösung:** Weitere Meta-Programmierung mit **Type Traits** und der **SFINAE**-Regel
- **SFINAE:** „*substitution failure is not an error*“
 - Wenn bei einer parametrisierte Funktion die Realisierung für einen bestimmten Datentypen nicht möglich ist, wird kein Fehler geworfen.
 - Stattdessen erzeugt der Compiler für diesen Fall keinen Code.
 - Wenn durch das Nicht-Erzeugen des Codes kein weiterer Fehler entsteht, wird Programm ausgeführt.

Ausblick: Meta-Programmierung



C++11 Type Traits Beispiel

```
#include <type_traits>
```

- Mit **Type Traits** werden zur Kompilierzeit Bedingungen über Daten-Typen evaluiert
- **`std::is_same<class T1, class T2>`** vergleicht zwei Datentypen auf Gleichheit und speichert das Ergebnis des Vergleichs in der Member-Variable **`::value`**

```
template<typename T>
std::string foo(T& param) {
    if (std::is_same<T, int>::value) {
        return "Integer"s;
    }
    else if (std::is_same<T, char>::value) {
        return "Character"s;
    }
    else {
        return "Other"s;
    }
}
```

```
int main() {
    int v1{ 3 };
    std::cout << foo(v1) << '\n';

    char v2{ 'c' };
    std::cout << foo(v2) << '\n';

    std::string v3{ "c"s };
    std::cout << foo(v3) << '\n';
}
```

Ausgabe:

```
Integer
Character
Other
```

Ausblick: Meta-Programmierung

SFINAE: Motivation



Sequence	std::vector<int>
Value	int

- Addition der Ganzzahlen
- Rückgabe als *int*

```
// Default-Verhalten für alle Typen
// außer den Überladenen
template<typename Sequence, typename Value>
Value sum(const Sequence& s, Value v) {
    for (auto x : s)
        v += x;
    return v;
}
```

Sequence	std::set<int>
Value	std::string

Sequence	Vector<std::string>
Value	std::string

- Interpretation der Strings als kodierte Zahlen
- Addition der Zahlen
- Rückgabe als *std::string*

```
// Überladung, wenn Value ein std::string ist
template<typename Sequence>
std::string sum(const Sequence& s, std::string v) {
    // Geh davon aus, dass im String ein Zahlenwert kodiert ist
    auto number = std::stol(v, nullptr, 0);
    // Geh davon aus, dass die Sequenz aus Strings besteht
    for (std::string x : s)
        number += std::stol(x, nullptr, 0);
    return std::to_string(number);
}
```

Problem: Sequence besteht bereits aus int-Werten.
std::stol funktioniert jedoch nur für Strings!

Ausblick: Meta-Programmierung

SFINAE: Motivation



Sequence	std::set<int>
Value	std::string

Sequence	Vector<std::string>
Value	std::string

Sequence	std::list<double>
Value	std::string

- **Ziel:** Unterscheide, welche Elemente Sequence verwaltet.

- Wenn sie aus Zahlenwerten besteht, soll eine andere Überladung ausgeführt werden als wenn sie aus Strings besteht!

```
// Überladung, wenn Sequence Zahlenwerte besitzt
template<typename Sequence>
std::string sum(const Sequence& s, std::string v) {
    // Geh davon aus, dass im String ein Zahlenwert kodiert ist
    auto number = std::stol(v, nullptr, 0);
    // Die Sequenz besteht aus Zahlen!
    for (auto x : s)
        number += x;
    return std::to_string(number);
}
```

```
// Überladung, wenn Sequence Strings besitzt
template<typename Sequence>
std::string sum(const Sequence& s, std::string v) {
    // Geh davon aus, dass im String ein Zahlenwert kodiert ist
    auto number = std::stol(v, nullptr, 0);
    // Die Sequenz besteht aus Strings!
    for (std::string x : s)
        number += std::stol(x, nullptr, 0);
    return std::to_string(number);
}
```

Ausblick: Meta-Programmierung



Type Traits für SFINAE

```
#include <type_traits>
```

- **C++11** Der ***type_traits*** Header stellt Möglichkeiten zur Verfügung, die weitere Spezialisierungen bei Funktionen ermöglichen
 - ***std::enable_if<bool B, class T>*** stellt einen public Member „*typedef type*“ der gleich T ist zur Verfügung, wenn der boolesche Ausdruck wahr ist. Ansonsten existiert der Member *typedef* nicht.
 - Kann als zusätzliches Funktionsargument, als Rückgabetyt oder Template-Parameter verwendet werden.
 - Bewerkstelligt, dass Funktionsüberladungen nicht zur Verfügung stehen, wenn die Bedingung nicht erfüllt ist

```
typename std::enable_if<std::is_same<std::string, typename Sequence::value_type>::value, std::string>::type
```

Value ist true, wenn die Datentypen übereinstimmen.

Erfordert, dass Sequence einen Member value_type besitzt.

Erfordert, dass std::enable_if einen Member type besitzt.

```
typename std::enable_if<std::is_arithmetic<typename Sequence::value_type>::value, std::string>::type
```

value ist true, wenn der Datentyp ein Zahlenwert ist.

Ausblick: Meta-Programmierung

Type Traits



```
#include <type_traits>
```

```
// Spezialisierung, wenn Sequenz Elemente von std::string besitzt
```

```
template<typename Sequence>
```

```
auto sum(const Sequence& s, std::string v)
```

Kann nur aufgerufen werden, wenn die Elemente in der Sequenz **Strings** sind.

```
-> typename std::enable_if<std::is_same<std::string, typename Sequence::value_type>::value, std::string>::type {
```

```
    // Geh davon aus, dass im String ein Zahlenwert kodiert ist
```

```
    auto number = std::stol(v, nullptr, 0);
```

```
    for (std::string x : s)
```

```
        number += std::stol(x, nullptr, 0);
```

```
    return std::to_string(number);
```

```
}
```

```
// Spezialisierung, wenn Sequenz aus Zahlenwerten besteht
```

```
template<typename Sequence>
```

```
auto sum(const Sequence& s, std::string v)
```

Kann nur aufgerufen werden, wenn die Elemente in der Sequenz **Zahlenwerte** sind.

```
-> typename std::enable_if<std::is_arithmetic<typename Sequence::value_type>::value, std::string>::type {
```

```
    // Geh davon aus, dass im String ein Zahlenwert kodiert ist
```

```
    typename Sequence::value_type number = std::stol(v, nullptr, 0);
```

```
    for (auto x : s)
```

```
        number += x;
```

```
    return std::to_string(number);
```

```
}
```

Berechnet den Zahlenwert auf dem Datentyp der Elemente in der Sequenz.

Ausblick: Meta-Programmierung

Type Traits



```
#include <type_traits>
```

```
template<typename Sequence>
auto sum(const Sequence& s, std::string v)
-> typename std::enable_if<std::is_same<std::string,
    typename Sequence::value_type>::value, std::string>::type
{
    auto number = std::stol(v, nullptr, 0);
    for (std::string x : s)
        number += std::stol(x, nullptr, 0);
    return std::to_string(number);
}

template<typename Sequence>
auto sum(const Sequence& s, std::string v)
-> typename std::enable_if<std::is_arithmetic<
    typename Sequence::value_type>::value, std::string>::type
{
    typename Sequence::value_type number = std::stol(v, nullptr, 0);
    for (auto x : s)
        number += x;
    return std::to_string(number);
}
```

```
int main() {
    std::list<std::string> strings = { "-10", "010", "0xF0" };
    std::cout << sum(strings, "0"s) << '\n';

    std::list<long> longs = { -10, 8, 240 };
    std::cout << sum(longs, "0"s) << '\n';

    std::list<double> doubles = { 7.2, 1.8, -4.6 };
    std::cout << sum(doubles, "0"s) << '\n';
}
```

Ausgabe:

```
238
238
4.400000
```

Ausblick: Meta-Programmierung



C++20 Concepts

```
#include <concepts>
```

- Klassen- und Funktions-Templates können mit einer **Einschränkung (Constraint)** versehen sein, die Anforderungen an Vorlagenargumente festlegt
 - Benannte Mengen von Constraints heißen **Concept**
 - Constraints sind eine **Folge von logischen Operatoren und Operanden** und können innerhalb von ***requires***-Ausdrücken oder direkt in einem Concept auftreten

```
template<class T, class U>  
concept Derived = std::is_base_of<U, T>::value;  
  
template<Derived<Base> T>  
void f(T); // T ist eingeschränkt durch Derived<T, Base>
```

```
template<Incrementable T>  
void f(T) requires Decrementable<T>;
```

requires-Ausdruck